МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №5

Специальность ПО-12

Выполнил

С. Э. Михновец,

студент группы ПО-12

Проверила

А. Д. Кулик,

преп. стаж. кафедры ИИТ,

«___k _____2026 г.

Брест 2026

Вариант: №15 (Отдел кадров)

1. Цель работы

Приобретение практических навыков проектирования реляционных баз данных, использования ORM (Object-Relational Mapping) для взаимодействия с данными на языке Python и создания RESTful API с использованием фреймворка FastAPI.

2. Задание по лабораторной работе (Вариант 15)

Реализовать информационную систему «Отдел кадров».
Требования:

- База данных должна содержать не менее 5 таблиц.
- Реализовать связи между таблицами (внешние ключи).
- Использовать SQLAlchemy для работы с БД.
- Реализовать CRUD-операции (Create, Read, Update, Delete).
- Создать эндпойнты FastAPI для каждой операции.

Структура реализованной БД:

1. departments (Отделы) — справочник подразделений компании.
2. positions (Должности) — справочник должностей с указанием оклада.
3. employees (Сотрудники) — основная таблица с данными сотрудников.
4. projects (Проекты) — список текущих проектов компании.
5. assignments (Назначения) — связующая таблица (многие-ко-многим) между сотрудниками и проектами.

3. Спецификации ввода-вывода программы

- **Методы ввода:**
  - POST: Передача JSON-объектов для создания новых записей (отделы, сотрудники).
  - GET: Получение данных (всех записей или конкретной записи по ID через Path-параметр).
  - PUT: Передача JSON-объекта для обновления существующей записи по ID.
  - DELETE: Удаление записи по идентификатору.
- **Формат вывода:**
  - Данные возвращаются в формате application/json.
  - В случае успеха возвращается объект или список объектов.
  - При ошибках возвращается стандартный JSON с деталями (например, {"detail": "Not Found"}).

4. Текст программы (основные фрагменты)

Ниже представлены ключевые части кода (модели и эндпойнты). Код отформатирован согласно стандартам **Black** и **Pylint**.

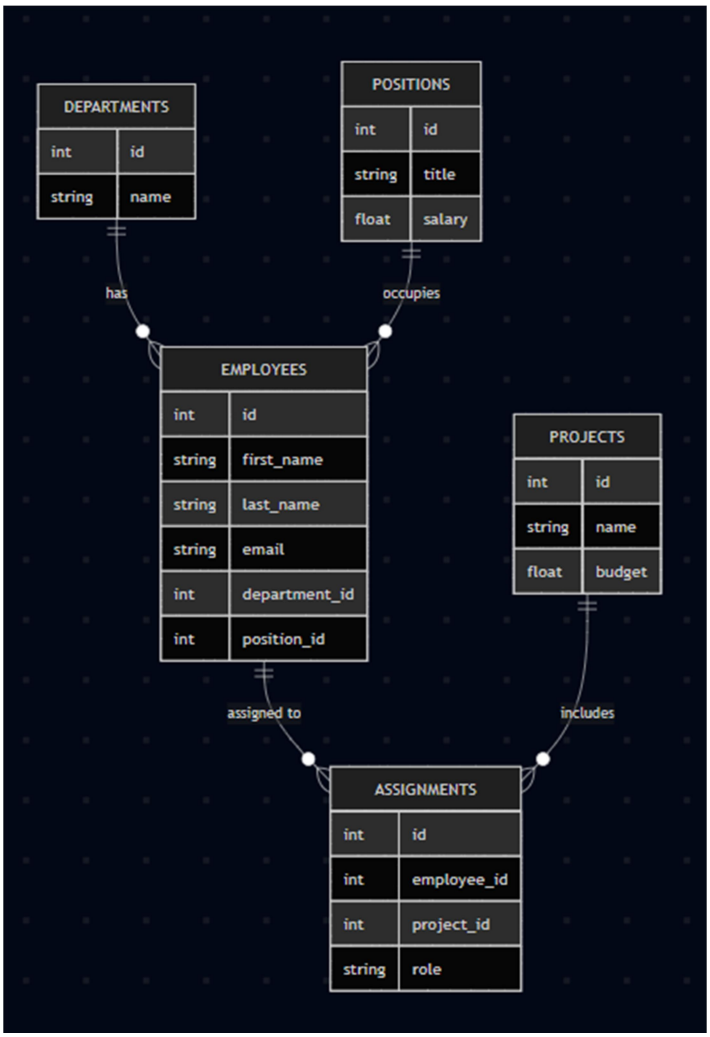
```
# Модели данных SQLAlchemy
class Employee(Base):
    __tablename__ = "employees"
    id = Column(Integer, primary_key=True, index=True)
```

```
    first_name = Column(String)
    last_name = Column(String)
    email = Column(String, unique=True)
    department_id = Column(Integer, ForeignKey("departments.id"))
    position_id = Column(Integer, ForeignKey("positions.id"))

# Схемы валидации Pydantic
class EmployeeCreate(BaseModel):
    first_name: str
    last_name: str
    email: str
    department_id: int
    position_id: int

# Эндпойнты FastAPI (CRUD)
@app.post("/employees/", response_model=EmployeeRead)
def create_employee(emp: EmployeeCreate, db: Session = Depends(get_db)):
    db_emp = Employee(**emp.model_dump())
    db.add(db_emp)
    db.commit()
    db.refresh(db_emp)
    return db_emp

@app.get("/employees/{emp_id}", response_model=EmployeeRead)
def read_employee(emp_id: int, db: Session = Depends(get_db)):
    return db.query(Employee).filter(Employee.id == emp_id).first()

@app.delete("/employees/{emp_id}")
def delete_employee(emp_id: int, db: Session = Depends(get_db)):
    db_emp = db.query(Employee).filter(Employee.id == emp_id).first()
    db.delete(db_emp)
    db.commit()
    return {"status": "deleted"}
```

5. Визуализация БД (Схема)

Схема реализована по принципу реляционных связей:

- Связь **1:N** между departments и employees.
- Связь **1:N** между positions и employees.
- Связь **M:N** между employees и projects через таблицу-посредник assignments.



6. Выводы

В результате выполнения лабораторной работы была спроектирована и реализована цифровая модель отдела кадров. Работа с **SQLAlchemy** позволила абстрагироваться от написания «сырых» SQL-запросов и использовать объектно-ориентированный подход. Фреймворк **FastAPI** обеспечил быструю разработку API с автоматической генерацией документации Swagger, что значительно упрощает тестирование и интеграцию фронтенд-части. Были освоены навыки валидации входящих данных с помощью Pydantic-схем и управления жизненным циклом сессий базы данных.